

내가 제안한 장기 프로젝트 관리+포트폴리오 저장: 프로젝트 볼트 프로그램을 만들건데 리스트와 조건문을 활용해 재미있는 로직을 짜줘. 그리고 리스트 조건문 반복문 함수/딕셔너리에 맞춰 내 컨셉의 프로그램이 어떻게 발전할 수 있을지 4단계 기능 확장 로드맵을 제안해줘.

좋아. 이건 **AI 협업 과정에 그대로 넣기 좋은 형태로 정리**해줄게.

네 컨셉인 **“장기 프로젝트 관리 + 포트폴리오 저장 = 프로젝트 볼트(Project Vault)”**에 맞춰,

먼저 **리스트+조건문 로직**을 재미있게 짜고, 그다음 **4단계 기능 확장 로드맵**까지 제안할게.

이 방향은 과제 안내에 나온 **1차 입출력 → 2차 리스트/조건문 → 3차 메뉴/반복문 → 4차 함수/딕셔너리** 구조에도 잘 맞는다.

1. 프로젝트 볼트용 리스트 + 조건문 로직 제안

프로젝트 볼트는 단순히 파일만 저장하는 프로그램이 아니라,

****“내 프로젝트가 지금 얼마나 잘 굴러가고 있는지”****와

****“나중에 포트폴리오로 꺼내 쓸 재료가 얼마나 쌓였는지”****를 함께 보여주는 방향이 재밌다.

그래서 각 프로젝트를 이런 정보로 관리하면 좋다.

프로젝트 1개에 담을 정보 예시

- 프로젝트명
- 분야
- 현재 진행률
- 저장한 자료 개수
- 중요도
- 최근 활동 공백일
- 회고 작성 여부
- 링크 등록 개수

2차 과제에서는 아직 딕셔너리 전 단계니까,

프로젝트 하나를 **리스트**로 저장하면 된다.

예시:

```
project = ["SPAD 연구", "연구", 45, 6, 9, 2, True, 3]
```

의미:

[프로젝트명, 분야, 진행률, 자료개수, 중요도, 공백일수, 회고작성여부, 링크개수]

2. 재미있는 핵심 로직

1) 프로젝트 준비도 점수 계산

단순 진행률만 보면 재미없으니까,
자료와 링크, 회고 여부까지 반영해서 **"프로젝트 준비도"**를 계산하게 하면 좋다.

예시 공식:

준비도 점수 = 진행률 + (자료개수 * 5) + (링크개수 * 3) + 중요도

회고를 작성했다면:

준비도 점수 + 10

즉,

- 진행률이 높을수록 좋고
 - 자료를 많이 쌓을수록 좋고
 - 관련 링크가 많을수록 좋고
 - 회고까지 작성하면 보너스를 받는 구조다
-

2) 프로젝트 상태 판정

조건문으로 상태를 이렇게 나누면 좋다.

if 준비도점수 >= 90:

 상태 = "순항"

elif 준비도점수 >= 70:

상태 = "진행 중"

elif 준비도점수 >= 50:

상태 = "정체"

else:

상태 = "위험"

이건 기본 뼈대고, 여기에 **공백일수** 조건을 추가하면 더 재밌다.

if 공백일수 >= 7:

상태 = "휴면"

즉,

아무리 자료가 많아도 **7일 이상 안 건드리면 휴면 프로젝트**가 되는 거다.

3) 특별 칭호 로직

이건 프로그램 맛을 살리는 부분이다.

아카이브 마스터

자료를 많이 쌓아둔 프로젝트

if 자료개수 >= 10 and 링크개수 >= 5:

칭호 = "아카이브 마스터"

폭풍 성장 프로젝트

짧은 공백 + 높은 진행률

if 진행률 >= 70 and 공백일수 <= 1:

칭호 = "폭풍 성장"

구조는 좋은데 멈춘 프로젝트

자료는 많은데 진행률이 낮음

if 자료개수 >= 8 and 진행률 < 40:

칭호 = "자료만 많은 프로젝트"

포트폴리오 완성형

회고까지 있고 진행률도 높고 자료도 충분함

if 진행률 >= 80 and 자료개수 >= 8 and 회고작성여부 == True:

 칭호 = "포트폴리오 완성형"

이런 복합 조건이 들어가면 단순 계산기가 아니라
진짜 "프로젝트 분석기"처럼 보인다.

3. 2차 과제용으로 쓰기 좋은 전체 로직 예시

2차에서는 여러 프로젝트를 리스트에 저장하고,
각 프로젝트마다 상태를 판정하는 구조로 가면 좋다.

예를 들면:

```
projects = [  
    ["SPAD 연구", "연구", 45, 6, 9, 2, True, 3],  
    ["자소서 정리", "취업", 80, 10, 8, 1, True, 5],  
    ["캡스톤 발표", "수업", 30, 4, 10, 8, False, 1]  
]
```

각 프로젝트에 대해:

- 준비도 점수 계산
- 상태 판정
- 특별 칭호 부여

이렇게 돌리면 된다.

예상 출력:

- SPAD 연구 → 준비도 94점 / 상태: 순항 / 칭호: 진행 중 핵심 프로젝트
- 자소서 정리 → 준비도 153점 / 상태: 순항 / 칭호: 포트폴리오 완성형
- 캡스톤 발표 → 준비도 63점 / 상태: 휴면 / 칭호: 구조는 좋으나 정체

이런 식으로 나오면 꽤 재밌고, 실사용성도 있다.

4. 프로젝트 볼트 4단계 기능 확장 로드맵

이제 네 컨셉을 리스트 → 조건문 → 반복문 → 함수/딕셔너리 흐름에 맞춰 자연스럽게 확장해보자.

1차: 입출력 + 변수

주제

프로젝트 기본 카드 생성기

구현 내용

- 사용자에게 프로젝트명, 분야, 현재 진행률, 자료 개수, 중요도를 입력받음
- 입력한 정보를 바탕으로 기초 준비도 점수를 계산
- 현재 프로젝트의 기본 요약 정보를 출력

핵심 포인트

- input()으로 데이터 받기
- int()로 진행률, 자료개수 형변환
- 간단한 산술연산으로 점수 계산
- f-string으로 보기 좋게 출력

1차 결과 예시

- 프로젝트명: SPAD 연구
- 분야: 연구
- 진행률: 35
- 자료개수: 4
- 중요도: 9
- 준비도 점수: 64

이 단계는 프로젝트 1개만 입력받아 분석하는 버전이다.

2차: 리스트 + 조건문

주제

다중 프로젝트 상태 분석기

구현 내용

- 프로젝트 정보를 리스트에 저장
- 여러 프로젝트를 한 번에 관리
- 준비도 점수와 공백일수 등을 기준으로 상태를 판정
- 조건문으로 특별 칭호 부여

핵심 포인트

- 프로젝트 1개를 리스트로 표현
- 여러 프로젝트를 다시 큰 리스트에 저장
- if-elif-else로 상태 판정
- and, or로 복합 칭호 로직 추가

2차에서 재밌는 기능

- 순항 / 정체 / 위험 / 휴면 판정
- 아카이브 마스터 / 폭풍 성장 / 포트폴리오 완성형 같은 칭호 부여

이 단계부터 프로그램이 확실히 살아난다.

3차: 반복문 + 메뉴 시스템

주제

프로젝트 볼트 운영 시스템

구현 내용

- 메뉴를 만들어 사용자가 원하는 기능을 반복 선택
- 프로젝트 추가

- 전체 프로젝트 조회
- 위험 프로젝트만 조회
- 포트폴리오화 가능한 프로젝트만 조회
- 종료 기능 추가

핵심 포인트

- while True로 무한 반복 메뉴 구성
- for문으로 전체 프로젝트 순회
- 번호 입력에 따라 기능 분기
- break로 종료

예시 메뉴

1. 프로젝트 등록
2. 전체 프로젝트 조회
3. 위험 프로젝트 확인
4. 포트폴리오 완성형 프로젝트 확인
5. 프로젝트 상태 분석
6. 종료

이 단계가 되면 진짜 소프트웨어처럼 보인다.

4차: 함수 + 딕셔너리

주제

프로젝트 볼트 최종 버전

구현 내용

- 기능별 코드를 함수로 분리
- 프로젝트 데이터를 리스트가 아닌 딕셔너리로 저장
- 각 항목 이름을 명확히 하여 가독성 향상

- 나중에 파일 저장 기능까지 붙이기 쉬운 구조로 개선

프로젝트 데이터 구조 예시

```
project = {  
    "name": "SPAD 연구",  
    "category": "연구",  
    "progress": 45,  
    "files": 6,  
    "priority": 9,  
    "inactive_days": 2,  
    "review": True,  
    "links": 3  
}
```

분리할 함수 예시

- add_project()
- calc_score()
- check_status()
- give_title()
- show_all_projects()
- show_risky_projects()

4차의 장점

- 코드가 길어져도 관리 쉬움
 - 프로젝트 항목이 늘어나도 수정 쉬움
 - 파일 저장/불러오기 확장 가능
 - 나중에 진짜 포트폴리오 관리 툴처럼 발전 가능
-

5. AI 협업 기록에 넣기 좋은 문장 형태

아래는 네가 과제에 바로 넣기 좋게 정리한 버전이다.

내용 1: 프로젝트 볼트 로직 설계

프롬프트 요약:

“내가 제안한 장기 프로젝트 관리+포트폴리오 저장 프로그램인 프로젝트 볼트를 만들 건데, 리스트와 조건문을 활용해 재미있는 로직을 짜줘.”

적용 내용:

AI가 제안한 방식에 따라 프로젝트를 [프로젝트명, 분야, 진행률, 자료개수, 중요도, 공백 일수, 회고작성여부, 링크개수] 형태의 리스트로 저장하고, 이를 바탕으로 준비도 점수를 계산하는 로직을 설계하였다. 또한 조건문을 활용해 프로젝트 상태를 '순항', '정체', '위험', '휴면'으로 판정하고, 자료 축적도와 진행 상태를 함께 반영하여 '아카이브 마스터', '폭풍 성장', '포트폴리오 완성형' 등의 특별 칭호를 부여하는 아이디어를 반영하였다.

내용 2: 4단계 확장 로드맵 설계

프롬프트 요약:

“리스트, 조건문, 반복문, 함수/딕셔너리에 맞춰 내 컨셉의 프로그램이 어떻게 발전할 수 있을지 4단계 기능 확장 로드맵을 제안해줘.”

적용 내용:

AI의 제안을 바탕으로 프로젝트 볼트를 4단계로 확장하는 로드맵을 정리하였다. 1차에서는 프로젝트 기본 정보를 입력받아 준비도 점수를 계산하고, 2차에서는 여러 프로젝트를 리스트에 저장해 조건문으로 상태와 칭호를 판정하도록 설계하였다. 3차에서는 반복문과 메뉴 시스템을 도입하여 프로젝트 등록, 조회, 위험 프로젝트 확인 기능을 추가하고, 4차에서는 함수와 딕셔너리를 활용해 프로젝트 데이터를 구조화하여 보다 실용적인 통합 관리 소프트웨어로 발전시키는 방향을 확정하였다.

원하면 다음 답변에서 내가 이걸 바탕으로 **2차 과제용 실제 파이썬 코드 초안**까지 바로 짜줄게.

파이썬 코드 v1.0 main.py 작성까지 해줘.

파이썬 코드 v1.0 - main.py

아래 코드를 그대로 main.py로 저장하면 된다.

```
# 프로젝트 볼트(Project Vault) v1.0
```

```
# 1차 과제용: 입출력 + 형변환 + 산술 연산 + 출력
```

```
print("==== 프로젝트 볼트(Project Vault) v1.0 ====")
```

```
project_name = input("프로젝트 이름을 입력하세요: ")
```

```
project_field = input("프로젝트 분야를 입력하세요: ")
```

```
final_goal = input("최종 목표를 입력하세요: ")
```

```
progress = int(input("현재 진행률(0~100)을 입력하세요: "))
```

```
file_count = int(input("저장한 자료 개수를 입력하세요: "))
```

```
link_count = int(input("등록한 링크 개수를 입력하세요: "))
```

```
priority = float(input("프로젝트 중요도(0.0~10.0)를 입력하세요: "))
```

```
archive_score = (file_count * 5) + (link_count * 3)
```

```
manage_score = progress + archive_score + priority
```

```
average_score = manage_score / 3
```

```
print("\n==== 프로젝트 기본 정보 ====")
```

```
print(f"프로젝트 이름: {project_name}")
```

```
print(f"프로젝트 분야: {project_field}")
```

```
print(f"최종 목표: {final_goal}")
```

```
print(f"현재 진행률: {progress}%")
```

```
print(f"저장한 자료 개수: {file_count}개")
```

```
print(f"등록한 링크 개수: {link_count}개")
```

```
print(f"프로젝트 중요도: {priority}")
```

```
print("\n==== 프로젝트 분석 결과 ====")
```

```
print(f"자료 준비 점수: {archive_score:.2f}")
```

```
print(f"초기 관리 점수: {manage_score:.2f}")
```

```
print(f"평균 분석 점수: {average_score:.2f}")
```

```
print(f"\n'{project_name}' 프로젝트의 기본 정보 입력과 초기 점수 계산이 완료되었습니다")
```

다.")

코드 한 줄 한 줄 핵심 설명

- `print("==== ... =====")`
프로그램 시작 화면을 출력한다.
- `project_name, project_field, final_goal`
문자열(str) 형태의 프로젝트 기본 정보를 입력받는다.
- `progress, file_count, link_count`
숫자 계산이 필요하므로 `int()`로 정수형 변환을 해준다.
- `priority`
중요도는 소수점 입력도 가능하게 하려고 `float()`를 사용했다.
- `archive_score = (file_count * 5) + (link_count * 3)`
저장한 자료와 링크 수를 이용해 자료 준비 점수를 계산한다.
- `manage_score = progress + archive_score + priority`
진행률, 자료 점수, 중요도를 더해서 초기 관리 점수를 만든다.
- `average_score = manage_score / 3`
전체 점수를 평균 형태로 한 번 더 보여주기 위한 값이다.
- 마지막 `print(f"...")`들
입력한 정보와 계산 결과를 보기 좋게 출력한다.
:.2f는 소수점 둘째 자리까지 출력하라는 뜻이다.

실행 예시

==== 프로젝트 볼트(Project Vault) v1.0 =====

프로젝트 이름을 입력하세요: SPAD 연구

프로젝트 분야를 입력하세요: 반도체 연구

최종 목표를 입력하세요: 발표 자료 초안 완성

현재 진행률(0~100)을 입력하세요: 35

저장한 자료 개수를 입력하세요: 4

등록한 링크 개수를 입력하세요: 3

프로젝트 중요도(0.0~10.0)를 입력하세요: 9.5

===== 프로젝트 기본 정보 =====

프로젝트 이름: SPAD 연구

프로젝트 분야: 반도체 연구

최종 목표: 발표 자료 초안 완성

현재 진행률: 35%

저장한 자료 개수: 4개

등록한 링크 개수: 3개

프로젝트 중요도: 9.5

===== 프로젝트 분석 결과 =====

자료 준비 점수: 29.00

초기 관리 점수: 73.50

평균 분석 점수: 24.50

'SPAD 연구' 프로젝트의 기본 정보 입력과 초기 점수 계산이 완료되었습니다.

원하면 바로 다음 단계로 **2차 과제용 main.py 코드(리스트 + 조건문 버전)**도 이어서 작성해줄게.

ㄱㄱ

Thought for 16s

좋아. 그럼 바로 ****2차 과제용 main.py****로 갈게.

이번 버전은 **입출력 + 리스트 + 조건문**에 맞춰서, 프로젝트 정보를 리스트에 저장하고 준비도 점수, 상태, 특별 칭호까지 출력하는 형태로 짰어. 2차 과제 방향인 **리스트 자료 구조 활용과 조건문 기반 상태 판정** 흐름에 맞춘 버전이다.

main.py

```
# 프로젝트 볼트(Project Vault) v2.0
```

```
# 2차 과제용: 입출력 + 리스트 + 조건문
```

```
print("===== 프로젝트 볼트(Project Vault) v2.0 =====")
```

```
# 1. 프로젝트 정보 입력
```

```
project_name = input("프로젝트 이름을 입력하세요: ")
```

```
project_field = input("프로젝트 분야를 입력하세요: ")
```

```
final_goal = input("최종 목표를 입력하세요: ")
```

```
progress = int(input("현재 진행률(0~100)을 입력하세요: "))
```

```
file_count = int(input("저장한 자료 개수를 입력하세요: "))
```

```
priority = int(input("프로젝트 중요도(0~10)를 입력하세요: "))
```

```
inactive_days = int(input("최근 활동 공백일 수를 입력하세요: "))
```

```
review_input = input("회고를 작성했나요? (yes/no): ")
```

```
link_count = int(input("등록한 링크 개수를 입력하세요: "))
```

```
# 2. 회고 작성 여부를 True / False로 변환
```

```
if review_input.lower() == "yes":
```

```
    review_done = True
```

```
else:
```

```
    review_done = False
```

```
# 3. 프로젝트 정보를 리스트에 저장
```

```
project = [
```

```
    project_name,
```

```
    project_field,
```

```
    final_goal,
```

```
    progress,
```

```
    file_count,
```

```
    priority,
```

```
    inactive_days,
```

```
    review_done,
```

```
    link_count
```

```
]
```

```
# 4. 전체 프로젝트 보관용 리스트에 추가
```

```
project_vault = []
```

```
project_vault.append(project)
```

```
# 5. 준비도 점수 계산
```

```
readiness_score = progress + (file_count * 5) + (link_count * 3) + priority
```

```
if review_done:
    readiness_score = readiness_score + 10
```

6. 프로젝트 상태 판정

```
if inactive_days >= 7:
    status = "휴면 프로젝트"
elif readiness_score >= 90:
    status = "순항"
elif readiness_score >= 70:
    status = "진행 중"
elif readiness_score >= 50:
    status = "정체"
else:
    status = "위험"
```

7. 특별 칭호 판정

```
title = "일반 프로젝트"
```

```
if progress >= 80 and file_count >= 8 and review_done:
    title = "포트폴리오 완성형"
elif file_count >= 10 and link_count >= 5:
    title = "아카이브 마스터"
elif progress >= 70 and inactive_days <= 1:
    title = "폭풍 성장 프로젝트"
elif file_count >= 8 and progress < 40:
    title = "자료만 많은 프로젝트"
```

8. 결과 출력

```
print("\n===== 저장된 프로젝트 정보 =====")
print(f"프로젝트 이름: {project_vault[0][0]}")
print(f"프로젝트 분야: {project_vault[0][1]}")
print(f"최종 목표: {project_vault[0][2]}")
print(f"현재 진행률: {project_vault[0][3]}%")
print(f"저장한 자료 개수: {project_vault[0][4]}개")
print(f"프로젝트 중요도: {project_vault[0][5]}")
```

```
print(f"최근 활동 공백일 수: {project_vault[0][6]}일")
print(f"회고 작성 여부: {project_vault[0][7]}")
print(f"등록한 링크 개수: {project_vault[0][8]}개")
```

```
print("\n===== 프로젝트 분석 결과 =====")
print(f"준비도 점수: {readiness_score}")
print(f"현재 상태: {status}")
print(f"특별 칭호: {title}")
```

```
print(f"\n'{project_name}' 프로젝트가 프로젝트 볼트에 저장되었습니다.")
```

이 코드에서 중요한 포인트

```
project = [...]
```

→ 프로젝트 하나의 정보를 **리스트**에 저장한 거야.

```
project_vault = []
```

```
project_vault.append(project)
```

→ 프로젝트 하나를 다시 큰 리스트에 넣어서, 나중에 여러 프로젝트를 저장할 수 있게 확장 가능하게 만든 거야.

```
if review_done:
```

→ 회고를 작성했으면 준비도 점수에 **보너스 10점**을 주는 조건문이야.

```
if inactive_days >= 7:
```

→ 점수가 높아도 너무 오래 손대지 않았으면 **휴면 프로젝트**로 먼저 판정하게 했어.
조건문 순서를 이렇게 뒤야 논리가 자연스러워.

and가 들어간 특별 칭호

→ 여러 조건을 동시에 만족할 때만 특별 칭호를 주게 해서, 단순 점수 계산보다 더 재밌는 로직이 되게 했어.

실행 예시

```
===== 프로젝트 볼트(Project Vault) v2.0 =====
```

```
프로젝트 이름을 입력하세요: SPAD 연구
```

```
프로젝트 분야를 입력하세요: 반도체 연구
```

```
최종 목표를 입력하세요: 발표 자료 초안 완성
```

```
현재 진행률(0~100)을 입력하세요: 82
```

```
저장한 자료 개수를 입력하세요: 9
```

프로젝트 중요도(0~10)를 입력하세요: 10

최근 활동 공백일 수를 입력하세요: 1

회고를 작성했나요? (yes/no): yes

등록한 링크 개수를 입력하세요: 6

===== 저장된 프로젝트 정보 =====

프로젝트 이름: SPAD 연구

프로젝트 분야: 반도체 연구

최종 목표: 발표 자료 초안 완성

현재 진행률: 82%

저장한 자료 개수: 9개

프로젝트 중요도: 10

최근 활동 공백일 수: 1일

회고 작성 여부: True

등록한 링크 개수: 6개

===== 프로젝트 분석 결과 =====

준비도 점수: 165

현재 상태: 순항

특별 칭호: 포트폴리오 완성형

'SPAD 연구' 프로젝트가 프로젝트 볼트에 저장되었습니다.